Algorithms:

Case study 1: Attendance Recovery Computation

Algorithm:

Step 1: start

Step 2: read classes attended=attended

Step 3: read classes conducted=conducted

Step 4: display “required 75% attendance”

Step 5: display 100*attended/conducted:.2f

Step 6: take to_attend=0

Step 7: take present=100*attended//conducted

Step 8: if present<75: then

Step 9: while present<75:

Step 10:take to_attend=to_attend+1

Step 11: take present=((to_attend+attended)/(conducted+to_attend))*100

Step 12: end while loop

Step 13: display to_attend

Step14: else display you have required percentage

Step 15: stop

Case study 2: Hospital Billing and Patient Classification

Algorithm:

Step 1: start

Step 2: take patients=[]

Step 3: read no of patients=n

Step 4: for I in range(n):

Step 5: display "Enter details for Patient",i+1

Step 6: read name of patient=name

Step 7: read consultation cost=consultation

Step 8: read diagnostic cost=diagnostic

Step 9: read treatment cost=treatment

Step10: take total=consulation+diagnostic+treatment

Step 11: if total>=10000: then

Step 12: discount=(total/100)*20

Step 13: elif total>=5000: then

Step 14: discount=(total/100)*10

Step 15: else discount=0

Step 16: take total_bill=total-discount

Step 17: if total_bill>=10000: then

Step 18: take category = "High Billing"

Step 19: e;if total_bill>=5000: then

Step 20: take category = "Medium Billing"

Step 21: else take category = "Low Billing"

Step 22: append([name,total,discount,total_bill,category]) into patients

Step 23: end for loop

Step 24: for j in patients

Step 25: display name, total, discount, total_bill, category

Step 26: end for lopp and diplay len(patients)

Step 27: stop

Case study 3: Library Borrowing and Fine Reconciliation

Algorithm:

Step 1: start

Step 2: take valid_books= ["book1","book2","book3","book4","book5"]

Step 3: take books=['book1'*3,'book2'*3,'book3'*3,'book4'*3,'book5'*3]

Step 4: display "available actions: take , return"

Step 5: read the user request= request

Step 5: if request==”take”: then

Step 6: read name of the book=name

Step 7: if name in books: then

Step 8: books.append(name)

Step 9: display "return within 5 days!"

Step 10: else: display "this is not available"

Step 11: stop inner if and outer if and elif request==”return”

Step 12: take name of book=name

Step 13: if name in valid_books: then

Step 14: books.append(name)

Step 15: stop inner if and take how many days=time

Step 16: if time>5:

Step 17: fine=50

Step 18: for I in range(5,time):

Step 19: fine=fine+50

Step 20: stop for loop

Step 21: else: fine=0

Step 22: display fine

Step 23: stop

Case study 4: Multi-Product Shopping Budget Constraint

Algorithm:

Step 1: start

Step 2: take products=[]

Step 3: take total=0

Step 4: read shopping budget=budget

Step 5: read number of products=n

Step 6: for i in range(n):

Step 7: read product name=name

Step 8: read price=price

Step 9: read quantity=quantity

Step 10: take cost=price*quantity

Step 11: take total=total+cost

Step 12: append [name, price, quantity, cost] into products

Step 13: end for loop

Step 14: for j in products:

Step 15: display j[0], "- Rs.", j[1], "x", j[2], "= Rs.", j[3]

Step 16: end for loop

Step 17: display "Total bill: Rs.", total

Step 18: if total <= budget: then

Step 19: display "Shopping is within your budget."

Step 20: else:

Step 21: display "Shopping exceeds your budget."

Step 22: display "Amount exceeded: ₹", total – budget

Step 23: stop

Case study 5: Multi-Stage Scholarship Shortlisting

Algorithm:

Step 1: start

Step 2: take details=[]

Step 3: read name and append to details

Step 4: read marks percentage and append to details

Step 5: read attendance percentage and append to details

Step 6: read income and append to details

Step 7: if details[1]>60 and details[2]>70: then

Step 8: if details[3]>10000: then

Step 9: take category='Merit-cum-Means scholarship'

Step 10: else:

Step 11: take category='Merit-Based scholarship'

Step 12: else:

Step 13: if details[3]>10000: then

Step 14: take category='Need-Based scholarship'

Step 15: else: Step 16: take category='no category'

Step 17: stop inner if-else and outer if-else

Step 18: display f'the student can apply for scholarship under {category}'

Step 19: stop

Case study 6: Password Policy Validation Engine

Algorithm:

Step 1: start

Step 1: start

Step 2: take password1=0

Step 3: while password1!="strong":

Step 4: take up=0, low=0, spe=0, num=0, length=0

Step 5: read password=password

Step 6: for i in password:

Step 7: if 48<=ord(i)<=57: then num+=1

Step 8: elif 65<=ord(i)<=90: then up+=1

Step 9: elif 97<=ord(i)<=122: then low+=1

Step 10: else: spe+=1

Step 11: take length+=1

Step 12: end for loop

Step 13: if length>8: then

Step 14: if length<20: then

Step 15: if num>0: then

Step 16: if low>0: then

Step 17: if up>0: then

Step 18: if spe>0: then

Step 19: display "your password is strong"

Step 20: take password1="strong"

Step 21: else: display "password doesn't have special characters", take password1="not strong"

Step 22: else: display "password doesn't have upper characters", take password1="not strong"

Step 23: else: display "password doesn't have lower characters", take password1="not strong"

Step 24: else: display "password doesn't have numbers", take password1="not strong"
Step 25: else: display "password is long", take password1="not strong"

Step 26: else: display "password is short", take password1="not strong"

Step 27: end while loop

Step 28: stop

Case study 7: Progressive Electricity Billing Algorithm:

Step 1: start

Step 1: start

Step 2: read number of households=n

Step 3: take bills=[]

Step 4: for i in range(n):

Step 5: display "Household", i + 1

Step 6: read household name=name

Step 7: read electricity units consumed=units

Step 8: if units <= 100: then bill=units*2

Step 9: elif units <= 200: then bill=units*3

Step 10: elif units <= 300: then bill=units*5

Step 11: else: bill=units*7

Step 12: take fixed_charge=50

Step 13: take total_bill=bill+fixed_charge

Step 14: append [name, units, total_bill] into bills

Step 15: end for loop

Step 16: for i in bills:

Step 17: display "Household:", i[0]

Step 18: display "Units consumed:", i[1]

Step 19: display "Energy charge: Rs.", i[2] - 50

Step 20: display "Fixed charge: Rs. 50"

Step 21: display "Total bill: Rs.", i[2]

Step 22: end for loop

Step 23: take highest=bills[0]

Step 24: for j in bills:

Step 25: if j[2]>highest[2]: then highest=j

Step 26: end for loop

Step 27: display "Highest bill:", highest[0]

Step 28: display "Amount: Rs.", highest[2]

Step 29: stop

Case study 8: Smart Canteen Order and Stock Management

Algorithm:

Step 1: start

Step 1: start

Step 2: take rice=20, eggs=30, noodles=10, chicken=2000

Step 3: take items=['fried rice','noodles','chicken rice']

Step 4: take special=['eggs','chicken']

Step 5: display "available items are:" and print items

Step 6: display "additional:" and print special

Step 7: take nxt_order=1

Step 8: while nxt_order!="no":

Step 9: if rice<1 or eggs<1 or noodles<1 or chicken<200: then

Step 10: take nxt_order=0

Step 11: display "out of stock"

Step 12: break while loop

Step 13: read order=order (in lowercase)

Step 14: read extra=extra (in lowercase)

Step 15: if order=="fried rice": then

Step 16: take rice=rice-1

Step 17: if extra=="egg": then eggs=eggs-1

Step 18: elif extra=="chicken": then chicken=chicken-200

Step 19: elif order=="chicken rice": then

Step 20: take rice=rice-1

Step 21: if extra=="egg": then eggs=eggs-1

Step 22: elif order=="noodles": then

Step 23: take noodles=noodles-1

Step 24: if extra=="egg": then eggs=eggs-1

Step 25: elif extra=="chicken": then chicken=chicken-200

Step 26: else: display "there no such type of item"

Step 27: read nxt_order

Step 28: end while loop

Step 29: stop

Case study 9: Smart Parking Fee Management

Algorithm:

Step 1: start

Step 1: start

Step 2: take A=["car", 'bike', 'truck', 'bicycle']

Step 3: display "available slots for vehicles", A

Step 4: read parking vehicle=parking_vehicle (lowercase and stripped)

Step 5: read time=time

Step 6: if parking_vehicle=='car': then maximum=10, fee=100

Step 7: elif parking_vehicle=='bike': then maximum=5, fee=50

Step 8: elif parking_vehicle=='truck': then maximum=15, fee=200

Step 9: else: maximum=0, fee=0

Step 10: if time>maximum: then

Step 11: display "you need to pay extra 1000"

Step 12: else:

Step 13: for i in range(time):

Step 14: fee=fee+20*(fee/100)

Step 15: end for loop

Step 16: display "your total fee is", int(fee)

Step 17: stop

Case study 10: University Scholarship Allocation

Algorithm:

Step 1: start

Step 1: start

Step 2: take students=[], marks=[], attendance=[], income=[]

Step 3: read number of students and loop that many times:

Step 4: read student name and append to students

Step 5: read marks percentage and append to marks

Step 6: read attendance percentage and append to attendance

Step 7: read income and append to income

Step 8: end for loop

Step 9: take eligible=[]

Step 10: for j in marks:

Step 11: if j>60: then

Step 12: take a=marks.index(j)

Step 13: append students[a] into eligible

Step 14: end for loop

Step 15: display "The eligible students are:"

Step 16: for k in eligible:

Step 17: display k (with end=" ")

Step 18: take b=students.index(k)

Step 19: if income[b]<80000: then display "with scholarship of 70%"

Step 20: elif income[b]>120000: then display "with scholarship of 50%"

Step 21: else: display "with scholarship of 30%"

Step 22: end for loop

Step 23: stop